MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

AI-BASED PACMAN GAME USING SEARCH ALGORITHMS

FISAC (ASSIGNMENT)

Submitted by

Vidushi Kumar
230906062

Joel Antony
230906418

Arjun R Pai
230906458

ABSTRACT

The project is focuses on development of AI-based Pacman game with the help of search algorithms. The primary goal is to create a smart agent that will be able to find its way in a maze and collect dots, avoiding ghosts effectively.

Other search methods are used including Breadth-First Search (BFS), Depth-First Search (DFS), and A* algorithm to find optimal paths. These algorithms are measured on performance measures such as the optimality of the path, computational efficiency and the ability to make decisions.

The project shows the possibility of using artificial intelligence methods in game settings and emphasizes the applicability of the methods in real-life applications like robotics, navigation systems, and decision-making procedures.

1. INTRODUCTION

The development of Artificial Intelligence (AI) has become an important aspect of contemporary technology, allowing machines to do the work that is usually carried out by human intelligence. Search algorithms are considered to be one of the most important fields of AI, and they are actively employed in problem-solving and pathfinding, as well as in decision-making. This project investigates the use of AI techniques in the traditional Pacman game setting.

Pacman is a famous arcade game the player has to follow a maze, gathering dots, and avoiding enemies (ghosts). The difficulty is to make the best decisions within real time and to cope with dynamic challenges. In this project, we will seek to substitute human control with an intelligent agent that can play the game independently by use of search algorithms.

We use uninformed and informed search methods. Breadth-First Search (BFS) and Depth-First Search (DFS) are uninformed methods that explore the environment without prior knowledge whereas informed methods like A star search use heuristics to enhance efficiency. These algorithms assist the agent to identify the optimal route to pick dots without colliding with ghosts.

The development of the game environment, modeling of the states, creation of rules and decision-making logic are also included in the project. The performance testing is done according to parameters like execution time, path length and success rate.

This contribution draws attention to the role of AI in the development of games, as well as gives an idea of the practical implementation, including autonomous vehicles, robotics navigation, intelligent systems, etc.

2. LITERATURE REVIEW

A number of studies have been used in the creation of AI-based agents to play games, particularly in the pacman world.

Koenig and Simmons (1995) examined the sophistication of the real-time search algorithms and their effectiveness in dynamic environment. Their efforts provided the basis of applying algorithms such as A star in games.

Structured exploration techniques introduced by Thrun and Schwartz (1995) enhanced the searching of complex environments by agents by finding patterns.

Nareyek (2001) investigated the use of genetic programming to create agents of Pacman which learns the best strategy by trial and error. This strategy presented game-based learning AI.

Hingston et al. (2004) and Lucas (2005) had done the evolutionary algorithms, which allowed the ghosts and pacman to evolve and learn better strategies during their evolution.

The theoretical basis of this project is given by Russell and Norvig (2010) who gave a detailed account of AI search algorithms.

In the scenario of playing pacman, Monte Carlo Tree Search (MCTS) was used by Fulda and Levine (2010) who achieved great performance.

Shleyfman and Stone (2011) developed probabilistic models based on POMDPs to manage uncertainty in the behavior of the ghosts.

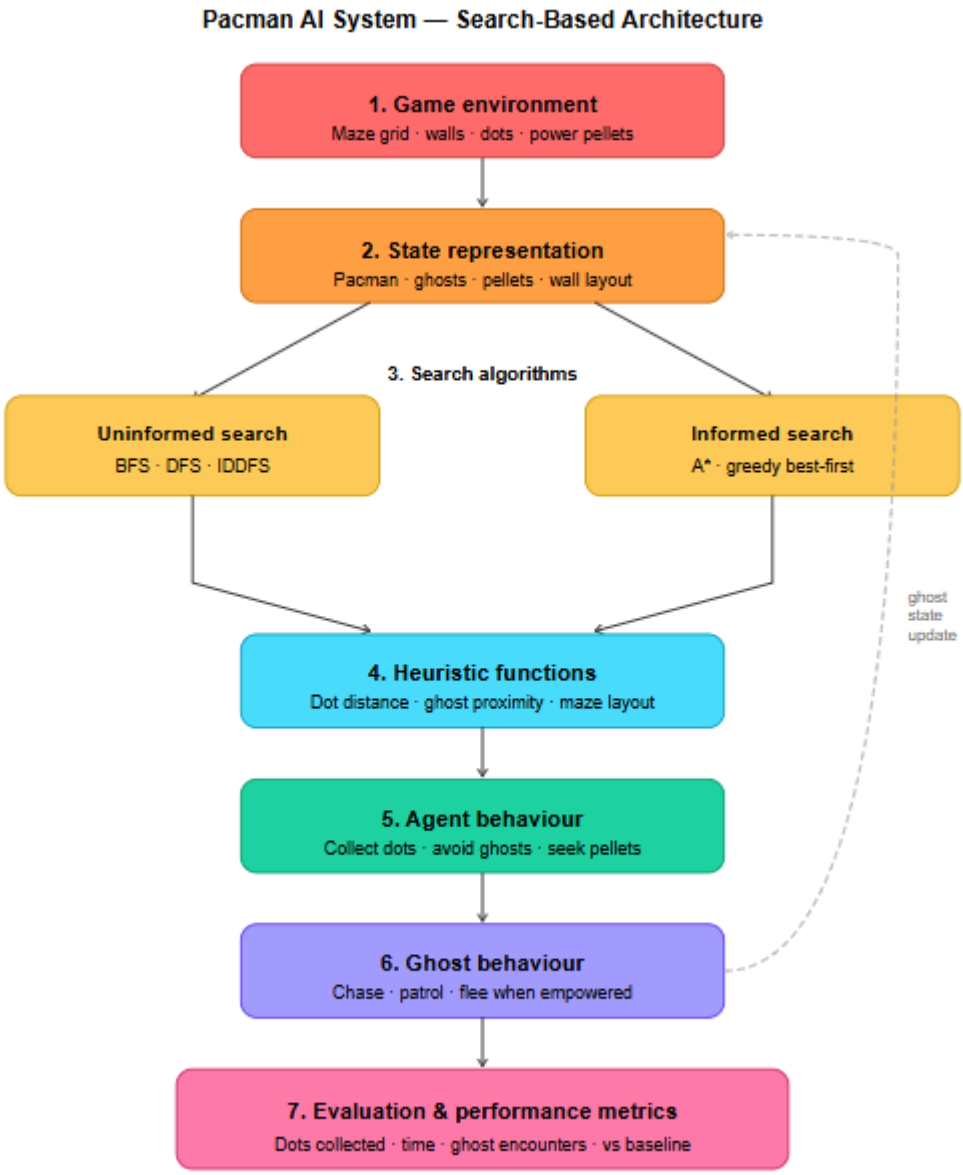
Recent studies by Gao and Mu (2020) trained agents by deep reinforcement learning using LSTM networks directly on game data. These works demonstrate how elementary search algorithms developed into elaborate AI methods, which represent the importance of this project in comprehending the basic concept of AI.

3. OBJECTIVES

To create AI-based game of Pacman with the help of search algorithms to find the optimal path using BFS, DFS, and A* search and make optimal choices.

To compare the performance of various search algorithms according to the parameters such as optimality, execution time, and memory consumption in a dynamic game environment.

4. METHODOLOGY



The design of a Pacman AI system using search techniques involves several key components and considerations to create an intelligent agent capable of autonomously playing the game. Below is an outline of the system architecture and design:

1. Game Environment: - The game environment consists of a maze-like grid where Pacman and ghosts navigate. - Each cell in the grid represents a location where Pacman, dots, power pellets, and ghosts can be positioned. - Walls and barriers define the maze structure, influencing the movement of Pacman and ghosts.
2. State Representation: - The state representation captures the current configuration of the game environment. - It includes the positions of Pacman, dots, power pellets, and

ghosts, as well as the layout of walls and barriers. - The state representation serves as the input to the search algorithms, guiding the decision-making process.

3. Search Algorithms: - Various search algorithms are employed to navigate the game environment and make decisions for Pacman. - Uninformed search algorithms, such as breadth-first search (BFS), depth-first search (DFS), and iterative deepening depth-first search (IDDFS), explore the state space without domain-specific knowledge. - Informed search algorithms, like A* search and its variants, utilize heuristic information to guide Pacman towards optimal paths while considering the positions of dots, power pellets, and ghosts.
4. Heuristic Functions: - Heuristic functions provide estimated measures of the cost or value associated with reaching a goal state. - These functions inform the search algorithms about the desirability of different actions or paths based on domain-specific knowledge. - Heuristics may consider factors such as the distance to the nearest dot or power pellet, the proximity of ghosts, and the layout of the maze.
5. Agent Behavior: - The AI agent controlling Pacman executes actions based on the decisions made by the search algorithms. - It determines the optimal direction for Pacman to move in each state, considering factors such as collecting dots, avoiding ghosts, and reaching the nearest power pellet.
6. Ghost Behavior: - Ghosts in the game exhibit different behaviors, such as chasing Pacman, patrolling specific areas, or fleeing when Pacman is empowered by a power pellet. - Ghost behavior influences Pacman's decision-making process, as it must anticipate and react to the movements of the ghosts to avoid being captured.
7. Evaluation and Performance Metrics: - The performance of the AI agent is evaluated based on metrics such as the number of dots collected, the time taken to complete the maze, and the number of encounters with ghosts. - Comparative analysis against baseline strategies and human performance may also be conducted to assess the effectiveness of the AI agent.

Code Screenshot:

pacman.py ✕ board.py

pacman.py > ...

```python

# Build Pac-Man from Scratch in Python with PyGame!!
import copy
from board import boards
import pygame
import math

pygame.init()

WIDTH = 900
HEIGHT = 950
screen = pygame.display.set_mode([WIDTH, HEIGHT])
timer = pygame.time.Clock()
fps = 60
font = pygame.font.Font('freesansbold.ttf', 20)
level = copy.deepcopy(boards)
color = 'gold'
PI = math.pi
player_images = []
for i in range(1, 5):
    player_images.append(pygame.transform.scale(pygame.image.load(f'assets/player_images/{i}.png'), (45, 45)))
blinky_img = pygame.transform.scale(pygame.image.load(f'assets/ghost_images/red.png'), (45, 45))
pinky_img = pygame.transform.scale(pygame.image.load(f'assets/ghost_images/pink.png'), (45, 45))
inky_img = pygame.transform.scale(pygame.image.load(f'assets/ghost_images/blue.png'), (45, 45))
clyde_img = pygame.transform.scale(pygame.image.load(f'assets/ghost_images/orange.png'), (45, 45))
spooked_img = pygame.transform.scale(pygame.image.load(f'assets/ghost_images/powerup.png'), (45, 45))
dead_img = pygame.transform.scale(pygame.image.load(f'assets/ghost_images/dead.png'), (45, 45))
player_x = 450
player_y = 663
direction = 0
blinky_x = 56
blinky_y = 58
blinky_direction = 0
inky_x = 440
inky_y = 388
inky_direction = 2
pinky_x = 440
pinky_y = 438
```

pacman.py ✕ board.py

pacman.py > ...

```python
score = 0
powerup = False
power_counter = 0
eaten_ghost = [False, False, False, False]
targets = [(player_x, player_y), (player_x, player_y), (player_x, player_y), (player_x, player_y)]
blinky_dead = False
inky_dead = False
clyde_dead = False
pinky_dead = False
blinky_box = False
inky_box = False
clyde_box = False
pinky_box = False
moving = False
ghost_speeds = [2, 2, 2, 2]
startup_counter = 0
lives = 3
game_over = False
game_won = False


class Ghost:
    def __init__(self, x_coord, y_coord, target, speed, img, direct, dead, box, id):
        self.x_pos = x_coord
        self.y_pos = y_coord
        self.center_x = self.x_pos + 22
        self.center_y = self.y_pos + 22
        self.target = target
        self.speed = speed
        self.img = img
        self.direction = direct
        self.dead = dead
        self.in_box = box
        self.id = id
        self.turns, self.in_box = self.check_collisions()
        self.rect = self.draw()

    def draw(self):
        if (not powerup and not self.dead) or (eaten_ghost[self.id] and powerup and not self.dead):
            screen.blit(self.img, (self.x_pos, self.y_pos))
        elif powerup and not self.dead and not eaten_ghost[self.id]:
            screen.blit(spooked_img, (self.x_pos, self.y_pos))
        else:
            screen.blit(dead_img, (self.x_pos, self.y_pos))
        ghost_rect = pygame.rect.Rect((self.center_x - 18, self.center_y - 18), (36, 36))
        return ghost_rect

    def check_collisions(self):
        # R, L, U, D
        num1 = ((HEIGHT - 50) // 32)
        num2 = (WIDTH // 30)
        num3 = 15
        self.turns = [False, False, False, False]
        if 0 < self.center_x // 30 < 29:
            if level[(self.center_y - num3) // num1][self.center_x // num2] == 9:
```

pacman.py board.py

pacman.py > ...

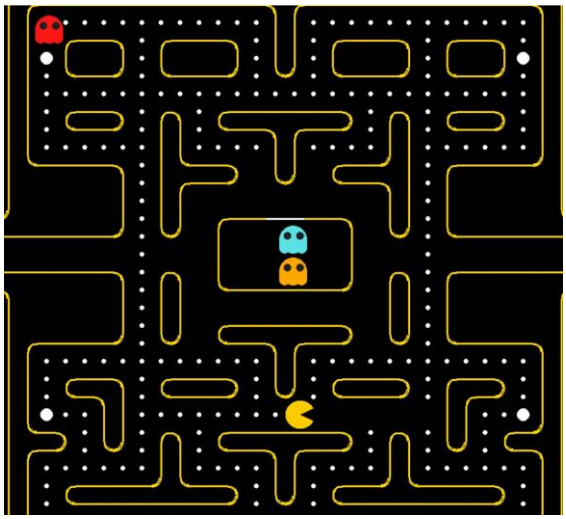
```python
        clyde_dead = True
        eaten_ghost[3] = True
        score += (2 ** eaten_ghost.count(True)) * 100

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            run = False
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RIGHT:
                direction_command = 0
            if event.key == pygame.K_LEFT:
                direction_command = 1
            if event.key == pygame.K_UP:
                direction_command = 2
            if event.key == pygame.K_DOWN:
                direction_command = 3
            if event.key == pygame.K_SPACE and (game_over or game_won):
                powerup = False
                power_counter = 0
                lives -= 1
                startup_counter = 0
                player_x = 450
                player_y = 663
                direction = 0
                direction_command = 0
                blinky_x = 56
                blinky_y = 58
                blinky_direction = 0
                inky_x = 440
                inky_y = 388
                inky_direction = 2
                pinky_x = 440
                pinky_y = 438
                pinky_direction = 2
                clyde_x = 440
                clyde_y = 438
                clyde_direction = 2
                eaten_ghost = [False, False, False, False]
                blinky_dead = False
                inky_dead = False
                clyde_dead = False
                pinky_dead = False
                score = 0
                lives = 3
                level = copy.deepcopy(boards)
                game_over = False
                game_won = False

        if event.type == pygame.KEYUP:
            if event.key == pygame.K_RIGHT and direction_command == 0:
                direction_command = direction
            if event.key == pygame.K_LEFT and direction_command == 1:
                direction_command = direction
            if event.key == pygame.K_UP and direction_command == 2:
                direction_command = direction
            if event.key == pygame.K_DOWN and direction_command == 3:
```

pacman.py board.py

board.py > ...

```python
# 0 = empty black rectangle, 1 = dot, 2 = big dot, 3 = vertical line,
# 4 = horizontal line, 5 = top right, 6 = top left, 7 = bot left, 8 = bot right
# 9 = gate
boards = [
[6, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 5],
[3, 6, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 5, 6, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 5, 3],
[3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3],
[3, 3, 1, 6, 4, 4, 4, 5, 1, 6, 4, 4, 4, 4, 5, 1, 3, 3, 1, 6, 4, 4, 4, 4, 5, 1, 6, 4, 4, 5, 1, 3, 3],
[3, 3, 2, 3, 0, 0, 3, 1, 3, 0, 0, 0, 3, 1, 3, 3, 1, 3, 3, 0, 0, 0, 3, 1, 3, 0, 0, 3, 2, 3, 3],
[3, 3, 1, 7, 4, 4, 8, 1, 7, 4, 4, 4, 8, 1, 7, 8, 1, 7, 4, 4, 4, 8, 1, 7, 4, 4, 8, 1, 3, 3],
[3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3],
[3, 3, 1, 6, 4, 4, 5, 1, 6, 5, 1, 6, 4, 4, 4, 4, 4, 4, 5, 1, 6, 5, 1, 6, 4, 4, 5, 1, 3, 3],
[3, 3, 1, 7, 4, 4, 8, 1, 3, 3, 1, 7, 4, 4, 5, 6, 4, 4, 8, 1, 3, 3, 1, 7, 4, 4, 8, 1, 3, 3],
[3, 3, 1, 1, 1, 1, 1, 1, 1, 3, 3, 1, 1, 1, 1, 3, 3, 1, 1, 1, 1, 3, 3, 1, 1, 1, 1, 1, 3, 3],
[3, 7, 4, 4, 4, 4, 5, 1, 3, 7, 4, 4, 5, 0, 3, 3, 0, 6, 4, 4, 8, 3, 1, 6, 4, 4, 4, 4, 8, 3],
[3, 0, 0, 0, 0, 0, 3, 1, 3, 6, 4, 4, 8, 0, 7, 8, 0, 7, 4, 4, 5, 3, 1, 3, 0, 0, 0, 0, 0, 3],
[3, 0, 0, 0, 0, 0, 3, 1, 3, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 3, 1, 3, 0, 0, 0, 0, 0, 3],
[8, 0, 0, 0, 0, 0, 3, 1, 3, 3, 0, 6, 4, 4, 9, 9, 4, 4, 5, 0, 3, 3, 1, 3, 0, 0, 0, 0, 0, 7],
[4, 4, 4, 4, 4, 4, 8, 1, 7, 8, 0, 3, 0, 0, 0, 0, 0, 0, 3, 0, 7, 8, 1, 7, 4, 4, 4, 4, 4, 4],
[0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 3, 0, 0, 0, 0, 0, 0, 3, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0],
[4, 4, 4, 4, 4, 4, 5, 1, 6, 5, 0, 3, 0, 0, 0, 0, 0, 0, 3, 0, 6, 5, 1, 6, 4, 4, 4, 4, 4, 4],
[5, 0, 0, 0, 0, 0, 3, 1, 3, 3, 0, 7, 4, 4, 4, 4, 4, 4, 8, 0, 3, 3, 1, 3, 0, 0, 0, 0, 0, 6],
[3, 0, 0, 0, 0, 0, 3, 1, 3, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 3, 1, 3, 0, 0, 0, 0, 0, 3],
[3, 0, 0, 0, 0, 0, 3, 1, 3, 3, 0, 6, 4, 4, 4, 4, 4, 4, 5, 0, 3, 3, 1, 3, 0, 0, 0, 0, 0, 3],
[3, 6, 4, 4, 4, 4, 8, 1, 7, 8, 0, 7, 4, 4, 5, 6, 4, 4, 8, 0, 7, 8, 1, 7, 4, 4, 4, 4, 5, 3],
[3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3],
[3, 3, 1, 6, 4, 4, 5, 1, 6, 4, 4, 4, 5, 1, 3, 3, 1, 6, 4, 4, 4, 5, 1, 6, 4, 4, 5, 1, 3, 3],
[3, 3, 1, 7, 4, 5, 3, 1, 7, 4, 4, 4, 8, 1, 7, 8, 1, 7, 4, 4, 4, 8, 1, 3, 0, 4, 8, 1, 3, 3],
[3, 3, 2, 1, 1, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3, 1, 1, 2, 3, 3],
[3, 7, 4, 5, 1, 3, 3, 1, 6, 5, 1, 6, 4, 4, 4, 4, 4, 4, 5, 1, 6, 5, 1, 3, 3, 1, 6, 4, 8, 3],
[3, 6, 4, 8, 1, 7, 8, 1, 3, 3, 1, 7, 4, 4, 5, 6, 4, 4, 8, 1, 3, 3, 1, 7, 8, 1, 7, 4, 5, 3],
[3, 3, 1, 1, 1, 1, 1, 1, 1, 3, 3, 1, 1, 1, 1, 3, 3, 1, 1, 1, 1, 3, 3, 1, 1, 1, 1, 1, 3, 3],
[3, 3, 1, 6, 4, 4, 4, 4, 8, 7, 4, 4, 5, 1, 3, 3, 1, 6, 4, 4, 8, 7, 4, 4, 4, 4, 4, 5, 1, 3, 3],
[3, 3, 1, 7, 4, 4, 4, 4, 4, 4, 4, 4, 4, 8, 1, 7, 8, 1, 7, 4, 4, 4, 4, 4, 4, 4, 4, 8, 1, 3, 3],
[3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3],
[3, 7, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 8, 3],
[7, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 8]
    ]
```

5. RESULTS & ANALYSIS



Analysis :

The algorithm uses a greedy directional heuristic instead of computing a complete path. Each ghost is assigned a dynamic target based on the game state, such as chasing the player or moving to a corner during power-up conditions.

Movement decisions are made locally at each intersection. The ghost compares its current position with the target and selects a direction that reduces the distance, while ensuring the path is not blocked.

Collision detection functions define the allowed directions by checking walls in the grid. This limits movement to valid paths and simplifies decision-making.

6. CONCLUSION

Implementing a Pacman game utilizing search algorithms like Depth-First Search (DFS), Breadth-First Search (BFS), and A* would provide a comprehensive exploration of pathfinding strategies. DFS, though simplistic, would embody Pacman's exploratory nature, traversing deep into the maze's branches. BFS, ensuring optimal paths, would enable Pacman to efficiently navigate towards pellets or evade ghosts, albeit at the cost of increased memory usage. A* would offer the best of both worlds, leveraging heuristics to guide Pacman towards the shortest path while considering various factors such as distance to targets and ghost avoidance. By comparing these algorithms' performances in terms of path length, execution time, and memory consumption, one could draw insights into their effectiveness in enhancing Pacman's gameplay experience and strategize accordingly.

7. CONTRIBUTION

Vidushi: Game Environment and Rendering

This part involved designing the game interface and visual elements. Created the game window using Pygame, implementing the board layout, drawing walls, pellets, and power-ups, and handling player animation. Functions such as board drawing and player rendering was developed.

Arjun: Player Control and Game Logic

Implemented keyboard input handling, movement control, collision detection with walls, and score updates. Functions like player movement, position checking, and collision handling with pellets and power-ups was developed.

Joel: Ghost AI and Movement Algorithm

Implemented of ghost behavior and decision-making.Designed the Ghost class, target selection logic, and movement functions for each ghost. The greedy heuristic-based movement, collision constraints, and state-based targeting (chase, scatter, power-up mode) is implemented

REFERENCES

- Koenig, S., & Simmons, R. (1995). Complexity analysis of real-time search in game playing. AAAI Workshop.
- Thrun, S., & Schwartz, A. (1995). Finding structure in exploration. AAAI Workshop.
- Nareyek, A. (2001). Learning to play Pac-Man using genetic programming. GECCO.
- Lucas, S. M. (2005). Evolutionary design of artificial Pac-Man. IEEE CEC.
- Hingston, S. G., Vamplew, P., & Garavan, H. (2004). Evolution and co-evolution of game strategies. GECCO.
- Fulda, N., & Levine, J. (2010). Monte Carlo Tree Search in Ms. Pac-Man. IEEE CIG.
- Shleyfman, A., & Stone, P. in 2011. Opponent modeling in Pac-Man using POMDPs. AAAI.
- Bellemare, M. G., et al. (2012). The Arcade Learning Environment. ICLR.
- Gao, Z., & Mu, J. (2020). Deep reinforcement learning for Pac-Man using LSTM networks. IEEE CoG In 2020